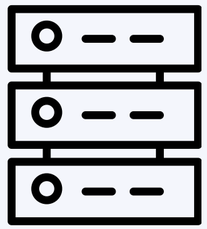
Engenharia de Computação - UNIVASF

Arthur Lincoln P. H Almeida &
Caio Benevides Pontes

Prolog Simulator



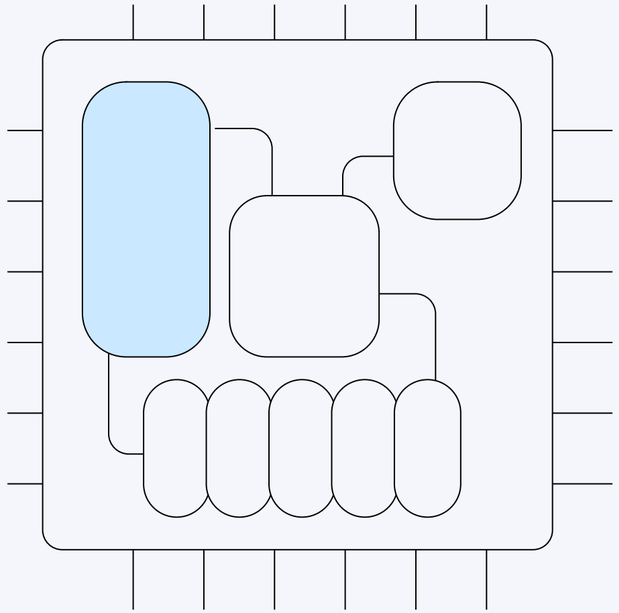
EM LINGUAGEM C



O que é Prolog?

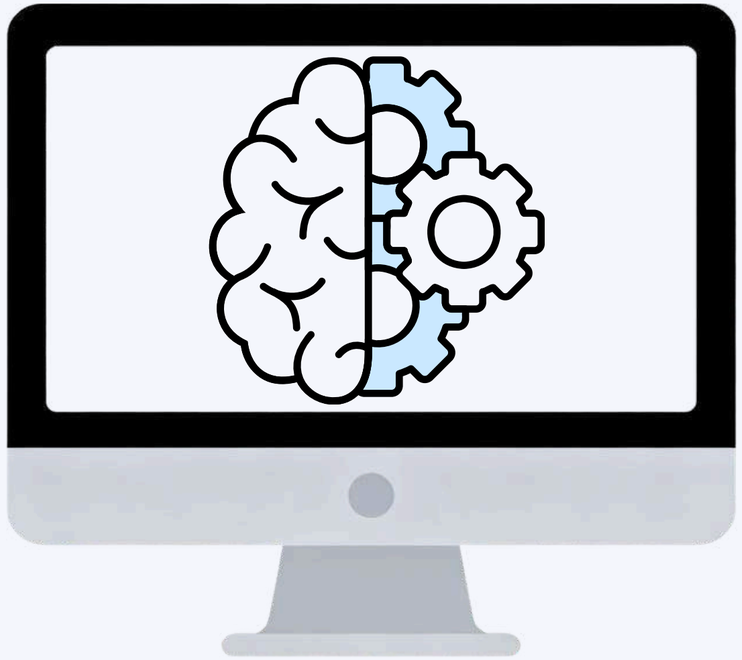
PROLOG É UMA LINGUAGEM DE PROGRAMAÇÃO BASEADA EM LÓGICA, NÃO EM INSTRUÇÕES SEQUENCIAIS COMO A MAIORIA DAS LINGUAGENS.

EM VEZ DE DIZER "FAÇA ISSO, DEPOIS AQUILO", VOCÊ DECLARA FATOS E REGRAS, E O PROLOG DESCOBRE SOZINHO AS RESPOSTAS A PARTIR DELES.



PROLOG É MAIS UTILIZADA EM ÁREAS QUE ENVOLVEM RACIOCÍNIO LÓGICO E PROCESSAMENTO DE LINGUAGEM NATURAL, COMO SISTEMAS DE INTELIGÊNCIA ARTIFICIAL.

TAMBÉM APARECE NO MEIO ACADÊMICO, ESPECIALMENTE EM CURSOS DE LÓGICA , POR SER UMA FERRAMENTA QUE PODE ENSINAR COMO MÁQUINAS PODEM "RACIOCINAR".



Por que Prolog em C?

PROLOG FUNCIONA DECLARANDO FATOS E REGRAS, E UM MOTOR BUSCA AS RESPOSTAS AUTOMATICAMENTE.

NOSSO PROJETO IMPLEMENTA UMA PEQUENA PROVA DE CONCEITO DESSE MOTOR, ALÉM DE AUTOMATIZAR A IMPLEMENTAÇÃO DE BANCOS DE DADOS POR MEIO DE ARQUIVOS DE TEXTO.



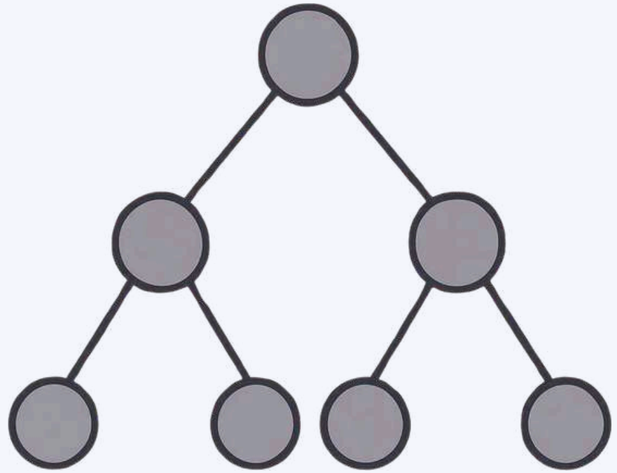


SWI Prolog

Sobre Estruturas de Dados

O PROJETO UTILIZA DUAS ESTRUTURAS COMPLEMENTARES:

- UMA ÁRVORE BINÁRIA DE BUSCA ONDE CADA NÓ TEM NO MÁXIMO DOIS FILHOS.
- UMA LISTA ENCADEADA, QUE ARMAZENA AS CLÁUSULAS DENTRO DE CADA NÓ DA BST NA ORDEM EM QUE FORAM DECLARADAS.





Sobre Estruturas de Dados

- A ÁRVORE BINÁRIA DE BUSCA É RESPONSÁVEL POR INDEXAR OS PREDICADOS POR NOME, PERMITINDO ENCONTRÁ-LOS RAPIDAMENTE SEM PRECISAR PERCORRER O BANCO INTEIRO.

- JÁ A LISTA ENCADEADA ARMAZENA AS CLÁUSULAS DE CADA PREDICADO NA ORDEM EM QUE FORAM DECLARADAS, POIS O MOTOR PRECISA TENTAR AS RESPOSTAS NESSA MESMA SEQUÊNCIA.

"Code-Snippets"

DEFINICÃO DE UM TERMO

```
typedef enum { CONSTANTE, VARIAVEL } TipoTermo;

typedef struct {
     TipoTermo tipo;
     char valor[MAX_NOME];
} Termo;
```

- TERMO É A MENOR UNIDADE DO SISTEMA. ELE PODE SER UMA CONSTANTE OU UMA VARIÁVEL. A DISTINÇÃO É FEITA PELO CAMPO TIPO, E O VALOR GUARDA O NOME.

"Code-Snippets"

DEFINICÃO DE UM TERMO

```
Termo termo_const(const char *v) {
    Termo t;
    t.tipo = CONSTANTE;
    strncpy(t.valor, v, MAX_NOME - 1);
    t.valor[MAX_NOME - 1] = '\0';
    return t;
}
```

- É UMA FUNÇÃO CONSTRUTORA. ELA CRIA UMA ESTRUTURA DO TIPO TERMO, DEFINE SEU TIPO E COPIA O TEXTO RECEBIDO COM SEGURANÇA PARA A MEMÓRIA.

NÓ DA ÁRVORE BINÁRIA(BST)

```
typedef struct NoBST {
      char nome[MAX_NOME];
      Clausula *clausulas;
      struct NoBST *esq;
      struct NoBST *dir;
 } NoBST;
```

- CADA NÓ DA ÁRVORE GUARDA O NOME DE UM PREDICADO COMO CHAVE, UM PONTEIRO PARA A LISTA DE CLÁUSULAS DAQUELE PREDICADO, E DOIS PONTEIROS PARA OS FILHOS ESQUERDO E DIREITO.

BUSCA NA ÁRVORE BINÁRIA

```
NoBST *bst_buscar(NoBST *raiz, const char *nome) {
    int cmp;
    if (!raiz) return NULL;
    cmp = strcmp(nome, raiz->nome);
    if (cmp < 0){
       return bst_buscar(raiz->esq, nome);
    }
    else if (cmp > 0){
       return bst_buscar(raiz->dir, nome);
    }
    return raiz; }
```

- DESCE A ÁRVORE COMPARANDO O NOME BUSCADO COM O NÓ ATUAL. SE FOR MENOR, VAI PARA A ESQUERDA. SE FOR MAIOR, VAI PARA A DIREITA. SE FOR IGUAL, ENCONTROU O PREDICADO.

UNIFICAÇÃO

1. Ambiente
2. Unificar
3. Evitando Colisões

- Se o banco de dados possui o fato “pai(joao, carlos)” e faz a pergunta “pai(joao,X)” a unificação é o mecanismo que compara as duas estruturas, percebe que elas são compatíveis e conclui que X deve ser carlos.

UNIFICAÇÃO

```
if (tc->tipo == VARIAVEL && vc == NULL) {
  amb_ligar(amb, tc->valor, vf ? vf : tf->valor);
}

else {
  if (!vc || !vf || strcmp(vc, vf) != 0) {
    return 0;
  }
}
```

- SE O LADO ESQUERDO É UMA VARIÁVEL LIVRE, ELA É LIGADA AO VALOR DO LADO DIREITO. POR EXEMPLO, AO COMPARAR HUMANO(X) COM HUMANO(CLARA), X RECEBE O VALOR CLARA. SE AMBOS OS LADOS SÃO CONCRETOS E DIFERENTES, A UNIFICAÇÃO FALHA.

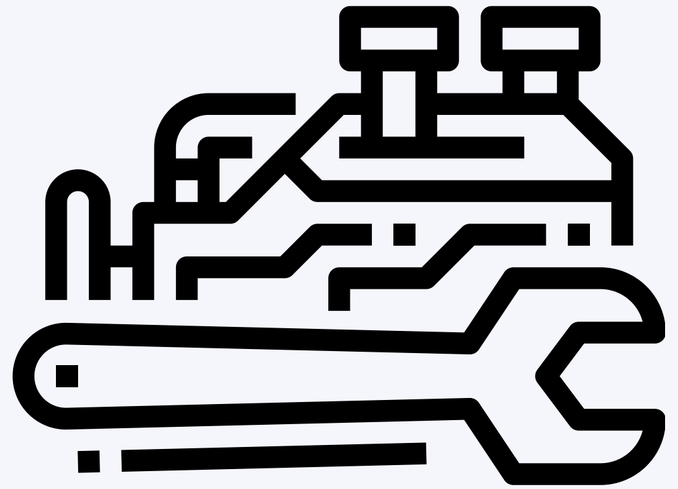
MOTOR COM BACKTRACKING

```
for (c = no->clausulas; c; c = c->prox) {
    novo_amb = amb_copiar(&amb);
    copia = *c;
    renomear_clausula(&copia, prof);
    if (!unificar(atual, &copia.cabeca, &novo_amb)){
       continue;
    }
    resolver(bd, objetivos + 1, n_obj - 1,
             novo_amb, prof + 1, vars_orig, n_vars_orig, encontrou);
}
```

- O LOOP PERCORRE TODAS AS CLÁUSULAS DO PREDICADO. PARA CADA UMA, COPIA O AMBIENTE ATUAL E TENTA UNIFICAR. SE FALHAR, O CONTINUE PASSA PARA A PRÓXIMA CLÁUSULA. ESSE É O BACKTRACKING. SE FUNCIONAR, CHAMA ‘RESOLVER’ RECURSIVAMENTE COM O RESTANTE DOS OBJETIVOS.

FUNÇÃO ‘RESOLVER’

- ‘RESOLVER’ É A FUNÇÃO PRINCIPAL DO MOTOR E É RECURSIVA.
- A CADA CHAMADA, ELA PEGA O PRIMEIRO OBJETIVO DA FILA E BUSCA O PREDICADO CORRESPONDENTE NA BST.
- O LOOP PERCORRE TODAS AS CLÁUSULAS DAQUELE PREDICADO TENTANDO UNIFICAR. SE A UNIFICAÇÃO FALHAR, O CONTINUE PASSA PARA A PRÓXIMA CLÁUSULA.
- ISSO É BACKTRACKING. SE FUNCIONAR, ‘RESOLVER’ É CHAMADA NOVAMENTE COM O RESTANTE DA FILA.



```
void resolver(BancoDados *bd, Atomo *objetivos, int n_obj,
              Ambiente amb, int prof,
              char vars_orig[][MAX_NOME], int n_vars_orig,
              int *encontrou) {
    Atomo *atual;
    NoBST *no;
    Clausula *c;
    int i, total;
    Ambiente novo_amb;
    Clausula copia;
    Atomo fila[MAX_SUBS + 8];

    if (prof > 30) return;

    if (n_obj == 0) {
        (*encontrou)++;
        printf("  Solucao %d:\n", *encontrou);
        imprimir_solucao(&amb, vars_orig, n_vars_orig);
        return;
    }
```

- bd → o banco de dados (BST)
- objetivos → fila de atomos a satisfazer
- n_obj → tamanho da fila
- amb → ambiente de variaveis (copia)
- prof → profundidade da recursao
- encontrou → contador de solucoes

- if (prof > 30) return;
  Limite de segurança contra recursão infinita.

- if (n_obj == 0)
  Caso base: fila vazia = todos os objetivos foram satisfeitos = solução encontrada.

```c
    atual = &objetivos[0];
    no    = bst_buscar(bd->raiz, atual->nome);

    if (!no) {
        printf("  (predicado '%s' desconhecido)\n", atual->nome);
        return;
    }

    for (c = no->clausulas; c; c = c->prox) {
        novo_amb = amb_copiar(&amb);
        copia    = *c;
        renomear_clausula(&copia, prof);

        if (!unificar(atual, &copia.cabeca, &novo_amb)) continue;

        if (copia.tipo == FATO) {
            resolver(bd, objetivos + 1, n_obj - 1,
                     novo_amb, prof + 1, vars_orig, n_vars_orig, encontrou);
        } else {
            total = copia.n_subs + (n_obj - 1);
            for (i = 0; i < copia.n_subs; i++)
                fila[i] = copia.subs[i];
            for (i = 1; i < n_obj; i++)
                fila[copia.n_subs + i - 1] = objetivos[i];
            resolver(bd, fila, total,
                     novo_amb, prof + 1, vars_orig, n_vars_orig, encontrou);
        }
    }
}
```

```
atual = &objetivos[0];
no    = bst_buscar(bd->raiz, atual->nome);

Pega o primeiro objetivo da fila e busca o predicado
correspondente na BST.

if (!no) return;
Se o predicado não existe no banco, para.

for (c = no->clausulas; c; c = c->prox)
Loop pelo backtracking: tenta cada cláusula do predicado.

novo_amb = amb_copiar(&amb);
Copia o ambiente antes de tentar — se falhar, a copia
é descartada e a próxima tentativa começa limpa.

if (!unificar(...)) continue;
Tenta unificar. Se falhar, pula para a próxima cláusula.

if (copia.tipo == FATO)
    resolver(objetivos + 1, n_obj - 1, ...)
Se é fato, remove o objetivo atual e resolve o restante.

else
    fila = subobjetivos da regra + restante da fila
    resolver(fila, total, ...)
Se é regra, expande os subobjetivos e coloca na frente
da fila antes de continuar resolvendo.
```

Agradecimentos:

- PROFESSORA DRA. ANA EMÍLIA DE MELO QUEIROZ;
- PROFESSOR DR. ROSALVO FERREIRA DE OLIVEIRA NETO;
- ANTHROPIC PBC.